

Week 1 - Class 1: Variables and Char

Why This Matters

In Python, creating a variable is as simple as writing `x = 5`. You don't tell Python what type `x` is; it figures it out at runtime. Later, you can even reassign `x = 'hello'` and Python will happily accept it.

C++ is very different. In C++, variables have a fixed type that is chosen at compile time and can never change. That type controls how much memory is reserved, what operations are legal, and how the compiler translates your code into machine instructions.

To see this difference clearly, we will start with the simplest C++ type: the `char`.

What Is a Variable in C++?

Think of a variable as a box in memory.

- In Python: the box only stores a reference to an object.
- In C++: the box stores the actual value (unless you explicitly use pointers or references).

Analogy:

Python: `x` is like a sticky note with the address of a house on it. The 'house' is the object.

C++: `x` is the house itself. Its layout is decided at compile time and cannot change.

The char Type

- `char` is the smallest integer type in C++.
- It always has size 1 byte.
- `CHAR_BIT` defines how many bits in a byte (usually 8).
- Typically `char` stores 8 bits -> 256 possible values.
- Signed `char` range: -128...127. Unsigned `char` range: 0...255.

Which one you get depends on your compiler.

Declaring and Initializing char

C++ offers several initialization styles:

```
char c1 = 'A'; // copy init
```

```
char c2('B'); // direct init
```

```
char c3{'C'}; // brace init (preferred)
```

Always initialize variables. Uninitialized variables contain garbage.

Example Program

```
#include <iostream>
#include <climits>
using namespace std;

int main() {
    char c1 = 'A';
    char c2{'B'};

    cout << "sizeof(char) = " << sizeof(char) << " byte(s)\n";
    cout << "CHAR_BIT = " << CHAR_BIT << " bits per byte\n";
    cout << "c1 = " << c1 << "   c2 = " << c2 << "\n";

    int code = (int)c1;
    cout << "ASCII of c1 = " << code << "\n";
}
```

C++14 Fundamentals - Student Edition

Sample Output

```
sizeof(char) = 1 byte(s)
CHAR_BIT = 8 bits per byte
c1 = A  c2 = B
ASCII of c1 = 65
```

ASCII Table (Partial)

```
+-----+-----+
| Char | Code |
+-----+-----+
| 'A'  | 65   |
| 'B'  | 66   |
| 'C'  | 67   |
| 'a'  | 97   |
| 'b'  | 98   |
| 'c'  | 99   |
| '0'  | 48   |
| '1'  | 49   |
| ' '  | 32   |
+-----+-----+
```

Memory Diagram

Variable: char c1 = 'A';

Memory (1 byte):

```
+-----+
| 01000001 | <- binary for ASCII 65 ('A')
+-----+
```

Common Mistakes

1. Thinking char is only for text. It's just a tiny integer.
2. Forgetting signedness. (char)200 may print as -56.
3. Using uninitialized variables leads to undefined behavior.

Exercises

1. Write a program that prints ASCII codes of 'A','B','C'.
2. Test whether char is signed or unsigned on your compiler.
3. Print CHAR_BIT.
4. Loop from 65 to 90 and print characters.

Project Tie-In

This class launches the Type Explorer Project. Today you explored char. In the next class, we will move on to int and explain what width really means.